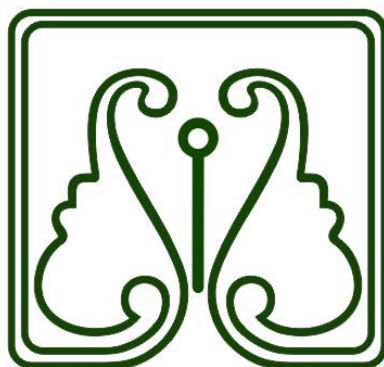


به نام خدا



دانشگاه گیلان

گزارش پروژه درس کامپایلر

طراحی و پیاده‌سازی تحلیل‌گر زبان ++Sea

شامل تحلیل لغوی، تحلیل نحوی، کنترل‌های معنایی و درخت نحو انتزاعی

نام دانشجو: امیرحسین خدیوی خانقاه

شماره دانشجویی: ۱۴۰۱۰۱۲۲۶۱۰۰۳

نام استاد: دکتر فیضی

نیم‌سال دوم تحصیلی ۱۴۰۵-۱۴۰۴

۲	چکیده
۲	۱. مقدمه و هدف پروژه
۳	۲. روند کلی نگارش و توسعه
۳	۳. محدوده عملکرد هر فاز
۳	۴. معماری پروژه و مسئولیت فایل‌ها
۴	۵. پیاده‌سازی فاز اول: تحلیل لغوی
۴	۶. پیاده‌سازی فاز دوم: Parser و کنترل ساختار
۴	۶-۱. سازه‌های پشتیبانی‌شده
۴	۶-۲. جدول نمادها و بررسی‌های معنایی
۵	۶-۳. درخت نحو انتزاعی (AST)
۵	۷. طراحی پوشه run و روش اجرای پروژه
۵	۷-۱. اجرای رابط گرافیکی
۵	۷-۲. اجرای محیط متنی تعاملی
۶	۸. مدیریت خروجی و خطا
۶	۹. آزمون، ارزیابی و کنترل کیفیت
۶	۱۰. چالش‌های اصلی و راه‌حل‌ها
۶	۱۱. جمع‌بندی

چکیده: در این پروژه یک کامپایلر آموزشی برای زبان ++Sea طراحی شد. فاز اول وظیفه شناسایی توکن‌ها و ثبت موقعیت آن‌ها را بر عهده دارد. در فاز دوم، ساختار برنامه بررسی شده، خطاهای نحوی و بخشی از خطاهای معنایی گزارش می‌شوند و درخت نحو انتزاعی نیز به‌صورت متنی، JSON و تصویر گرافیکی تولید می‌گردد. برای سهولت استفاده، اجرای پروژه هم از طریق رابط گرافیکی و هم از طریق محیط متنی تعاملی فراهم شده است؛ در هر دو روش کاربر تنها فایل و فاز موردنظر را انتخاب می‌کند و تمام خروجی‌ها در پوشه output ذخیره می‌شوند.

۱. مقدمه و هدف پروژه

هدف اصلی پروژه، پیاده‌سازی مرحله‌به‌مرحله بخش‌های پایه یک کامپایلر برای زبان آموزشی ++Sea بود. روند توسعه به‌گونه‌ای انتخاب شد که هر بخش مستقل، قابل آزمون و قابل استفاده در مرحله بعد باشد. ابتدا متن برنامه به توکن تبدیل شد؛ سپس توکن‌ها برای تشخیص ساختارهای زبان در اختیار Parser قرار گرفتند و در نهایت اطلاعات معنایی و درخت ساختار برنامه تولید شد.

در طراحی پروژه دو اصل در نظر گرفته شد: نخست، جداسازی مسئولیت‌ها؛ به این معنا که Parser، Lexer، جدول نمادها، AST و رابط اجرا هرکدام در فایل جداگانه قرار داشته باشند. دوم، حفظ قابلیت استفاده؛ بنابراین کاربر برای اجرای پروژه مجبور به نوشتن دستورهای پیچیده نیست و می‌تواند با اجرای فایل مناسب، ورودی و فاز را از داخل برنامه انتخاب کند.

۲. روند کلی نگارش و توسعه

1. مطالعه مشخصات زبان ++Sea و استخراج کلمات کلیدی، عملگرها، انواع داده و ساختارهای نحوی مورد نیاز.
2. تعریف انواع توکن و کلاس Token برای نگهداری نوع، مقدار، شماره خط و شماره ستون.
3. پیاده‌سازی Lexer و آزمایش آن با ورودی‌های صحیح، توضیحات، رشته‌ها، اعداد و کاراکترهای نامعتبر.
4. طراحی Parser به صورت تابع‌محور؛ برای هر سازه مانند کلاس، تابع، شرط، حلقه، تعریف متغیر و عبارت، یک مسیر تجزیه مشخص در نظر گرفته شد.
5. افزودن جدول نمادها و کنترل‌های معنایی مانند تعریف تکراری نام‌ها در محدوده یکسان.
6. ساخت AST و افزودن خروجی متنی، DOT، JSON و تصویر گرافیکی با Graphviz.
7. ایجاد پوشه run برای اجرای یکپارچه پروژه از طریق GUI و محیط متنی تعاملی.
8. نوشتن تست‌های خودکار و بررسی سناریوهای موفق و ناموفق با pytest.

۳. محدوده عملکرد هر فاز

خروجی	وظیفه اصلی	فاز
فهرست توکن‌ها همراه نوع، مقدار، خط و ستون	شناسایی توکن‌ها بدون اجرای Parser	فاز ۱
گزارش Parser، خطاها و AST	تحلیل نحوی و کنترل ساختار برنامه	فاز ۲
جدول نمادها، DOT، JSON و تصویر AST	کنترل‌های معنایی و نمایش گرافیکی	بخش تکمیلی

۴. معماری پروژه و مسئولیت فایل‌ها

معماری پروژه لایه‌ای و ماژولار است. فایل‌های داخل src منطق کامپایلر را تشکیل می‌دهند، فایل‌های samples ورودی‌های نمونه را نگهداری می‌کنند، tests برای آزمون خودکار استفاده می‌شود و پوشه run تنها مسئول اجرای کاربرپسند پروژه است. تمام فایل‌های تولیدشده نیز در output قرار می‌گیرند تا کدهای اصلی با خروجی‌ها مخلوط نشوند.

فایل‌ها و مسئولیت	بخش
lexer.py، token_type.py، sea_token.py؛ تعریف انواع توکن و استخراج آن‌ها از متن	تحلیل لغوی
parser.py؛ بررسی ترتیب توکن‌ها و تشخیص سازه‌های زبان	تحلیل نحوی
symbol_table.py؛ مدیریت محدوده‌ها و کنترل تعریف‌های تکراری	اطلاعات معنایی
ast_nodes.py؛ نگهداری، چاپ، ذخیره و رسم AST	درخت برنامه
run/main.py و run/common.py، run/gui.py، run/terminal.py	اجرای پروژه
test_parser.py و test_lexer.py؛ بررسی رفتار صحیح و خطاها	آزمون

۵. پیاده‌سازی فاز اول: تحلیل لغوی

- Lexer متن ورودی را از ابتدا تا انتها پیمایش می‌کند. در هر موقعیت، بر اساس کاراکتر جاری تصمیم می‌گیرد که با فاصله، توضیح، شناسه، عدد، رشته، عملگر یا جداکننده روبرو است. پس از تشخیص، یک شیء Token ساخته می‌شود و محل شروع آن نیز ثبت می‌گردد. نگهداری شماره خط و ستون باعث می‌شود گزارش خطا در مراحل بعد دقیق و قابل فهم باشد.
- فاصله‌ها و خطوط خالی نادیده گرفته می‌شوند، ولی شماره خط و ستون به‌روزرسانی می‌شود.
 - شناسه‌ها پس از خواندن با فهرست کلمات کلیدی مقایسه می‌شوند.
 - اعداد صحیح، اعشاری و پسوند‌های نوع داده به‌صورت جداگانه تشخیص داده می‌شوند.
 - رشته‌ها تا علامت پایان خوانده می‌شوند و رشته بسته‌نشده به‌عنوان خطا گزارش می‌شود.
 - عملگرهای یک‌کاراکنری و چندکاراکنری با اولویت طول بیشتر بررسی می‌شوند.
 - در پایان ورودی، توکن EOF برای اطلاع Parser اضافه می‌شود.
- نمونه شکل خروجی فاز اول:

```
KEYWORD(class)      Line 1, Column 1
IDENTIFIER(MyClass) Line 1, Column 7
KEYWORD(begin)      Line 1, Column 15
INTEGER_LITERAL(10) Line 2, Column 18
```

۶. پیاده‌سازی فاز دوم: Parser و کنترل ساختار

Parser فهرست توکن‌ها را دریافت می‌کند و با حرکت مرحله‌ای روی آن‌ها، ساختار برنامه را می‌سازد. برای هر ساختار اصلی یک متد مجزا در نظر گرفته شده است؛ برای مثال تجزیه کلاس، تابع، پارامترها، بلوک، تعریف متغیر، شرط، حلقه، دستور return و عبارت‌ها در مسیرهای جدا انجام می‌شود. این روش باعث شده است خطاها در همان نقطه‌ای که توکن مورد انتظار وجود ندارد، با پیام و موقعیت دقیق گزارش شوند.

۶-۱. سازه‌های پشتیبانی‌شده

- تعریف کلاس و اعضای آن، تعریف تابع و پارامترهای ورودی، و الزام وجود تابع اصلی برنامه.
- تعریف متغیر برای انواع پایه و نوع‌های کلاسی، همراه مقداردهی اولیه اختیاری.
- دستورهای شرطی، شاخه‌های else-if و else، و بلوک‌های begin/end.
- حلقه‌های while و for، دستورهای انتساب، افزایش و کاهش، و return.
- عبارت‌های حسابی، مقایسه‌ای و منطقی با رعایت تقدم عملگرها، فراخوانی تابع و دسترسی به اعضا.

۶-۲. جدول نمادها و بررسی‌های معنایی

جدول نمادها برای نگهداری نام کلاس‌ها، توابع، پارامترها و متغیرها استفاده می‌شود. هر بلوک می‌تواند محدوده مستقلی داشته باشد؛ بنابراین یک نام در محدوده‌های متفاوت قابل استفاده است، اما تعریف تکراری همان نام در محدوده یکسان خطا محسوب می‌شود. بررسی‌های تکمیلی پروژه شامل تشخیص کلاس یا تابع تکراری، متغیر تکراری، تعداد نامعتبر تابع main و وابستگی چرخه‌ای میان کلاس‌ها است.

۶-۳. درخت نحو انتزاعی (AST)

همزمان با تحلیل نحوی، برای اجزای مهم برنامه گره AST ساخته می‌شود. هر گره دارای نوع، ویژگی‌ها، فرزندان و در صورت نیاز موقعیت خط و ستون است. این درخت جزئیات کم‌اهمیت نحوی مانند بعضی جداکننده‌ها را حذف می‌کند و ساختار واقعی برنامه را نشان می‌دهد. خروجی AST در چهار قالب ارائه می‌شود:

- نمایش درختی متنی برای مطالعه سریع ساختار برنامه.
- فایل JSON برای استفاده ماشین‌ی و توسعه مراحل بعد.
- فایل DOT برای نگهداری تعریف گراف.
- تصویر PNG با استفاده از Graphviz برای نمایش بصری گره‌ها و ارتباط آن‌ها.

```
Program
ClassDeclaration(name="MyClass")
FunctionDeclaration(name="main")
VariableDeclaration(type="int", name="x")
ReturnStatement
```

۷. طراحی پوشه run و روش اجرای پروژه

برای جلوگیری از پراکندگی فایل‌های اجرایی، تمام امکانات مربوط به اجرا در پوشه run قرار گرفت. منطق مشترک در common.py نوشته شده است تا GUI و محیط متنی از یک تابع واحد استفاده کنند و نتیجه هر دو روش یکسان باشد. فایل main.py اجرای پیش‌فرض رابط گرافیکی را انجام می‌دهد.

فایل	نقش
run/common.py	اجرای مشترک فاز ۱ و ۲، نام‌گذاری و ذخیره خروجی‌ها
run/gui.py	تنها رابط گرافیکی پروژه برای انتخاب فایل، فاز و مشاهده نتیجه
run/terminal.py	محیط متنی تعاملی؛ کاربر فقط فایل و شماره فاز را وارد می‌کند
run/main.py	نقطه شروع اجرای GUI

۷-۱. اجرای رابط گرافیکی

1. کاربر فایل run/main.py را اجرا می‌کند و پنجره برنامه باز می‌شود.
2. فایل ++Sea از طریق دکمه Open Input File انتخاب می‌شود یا کد مستقیماً در ویرایشگر نوشته می‌شود.
3. حالت Phase 1 - Lexer Only یا Phase 2 - Full Analysis انتخاب می‌شود.
4. با زدن Analyze، نتیجه در پنل سمت راست نمایش داده شده و فایل‌های خروجی ذخیره می‌شوند.
5. در فاز دوم، کاربر می‌تواند تصویر AST و پوشه خروجی را با دکمه‌های برنامه باز کند.

۷-۲. اجرای محیط متنی تعاملی

فایل run/terminal.py نیز مستقیماً قابل اجرا است. در این روش کاربر هیچ دستور پایتون یا آرگومان خط فرمانی نمی‌نویسد. برنامه پس از اجرا مسیر فایل ورودی را درخواست می‌کند، سپس انتخاب فاز ۱ یا ۲ را می‌پرسد و در پایان مسیر فایل‌های ساخته‌شده را نمایش می‌دهد.

۸. مدیریت خروجی و خطا

پوشه output تنها محل ذخیره فایل‌های تولیدی است. این تصمیم باعث می‌شود پاک‌کردن خروجی‌ها روی فایل‌های src اثر نگذارد و ساختار پروژه منظم باقی بماند. در صورت وقوع خطا، پیام در GUI یا ترمینال نمایش داده شده و یک گزارش متنی زمان‌دار نیز در output ثبت می‌شود. نام خروجی‌ها بر اساس نام فایل ورودی ساخته می‌شود تا نتایج چند آزمایش با یکدیگر اشتباه نشوند.

۹. آزمون، ارزیابی و کنترل کیفیت

برای اطمینان از صحت عملکرد، تست‌های خودکار با pytest نوشته شدند. تست‌های Lexer نوع و ترتیب توکن‌ها، موقعیت خط و ستون، رشته‌ها، اعداد، توضیحات و خطاهای لغوی را بررسی می‌کنند. تست‌های Parser نیز برنامه‌های معتبر، خطاهای نحوی، وجود main، تعریف‌های تکراری و بخش‌های مربوط به AST را پوشش می‌دهند.

- آزمون‌های موفق، تحلیل صحیح کلاس، تابع main، متغیر، شرط و حلقه را بررسی می‌کنند.
- آزمون‌های خطا، کاراکتر نامعتبر، رشته بسته‌نشده، نبود نمادهای ضروری و تعریف‌های تکراری را پوشش می‌دهند.
- آزمون‌های AST، ایجاد ساختار درخت، JSON و تصویر گرافیکی را کنترل می‌کنند.

۱۰. چالش‌های اصلی و راه‌حل‌ها

تفکیک فازها: در فاز ۱ اجرای برنامه پس از تولید توکن‌ها پایان می‌یابد تا Parser روی ورودی‌های لغوی اجرا نشود. هماهنگی خروجی‌ها: منطق مشترک در common.py متمرکز شد تا GUI و محیط متنی نتیجه یکسان تولید کنند. نمایش AST: خروجی متنی و JSON با فایل DOT و تصویر Graphviz تکمیل شد. ایمنی پروژه: کدها در src و run و همه فایل‌های تولیدی فقط در output نگهداری می‌شوند.

۱۱. جمع‌بندی

پروژه حاضر یک چارچوب کامل و منظم برای آموزش مراحل ابتدایی ساخت کامپایلر ارائه می‌دهد. تحلیل لغوی، تحلیل نحوی، جدول نمادها، کنترل‌های معنایی، AST و رابط‌های اجرا به‌صورت مستقل ولی هماهنگ پیاده‌سازی شده‌اند. خروجی‌های دقیق و تست‌های خودکار نیز امکان بررسی و توسعه پروژه را فراهم می‌کنند. در وضعیت فعلی، پروژه نیازمندی‌های اصلی دو فاز و بخش‌های تکمیلی را پوشش می‌دهد و می‌تواند در آینده با بررسی کامل سازگاری نوع‌ها، تولید کد میانی، بهینه‌سازی و تولید کد مقصد گسترش یابد.

نتیجه نهایی: کاربر با اجرای یک فایل، انتخاب ورودی و تعیین فاز می‌تواند کل فرایند تحلیل را انجام دهد؛ بدون آنکه نیاز باشد جزئیات داخلی Lexer، Parser یا شیوه ذخیره خروجی‌ها را بداند.